

PYTHON ALARM CLOCK

Project Report

Build Your Own Project

Prepared by: Zeion jain

Registration No.:26BAI10975

Course / Subject: Cse

Institution: VIT Bhopal University

Academic Year: 2026–27

1. Introduction

The Python Alarm Clock is a small command-line application made to handle everyday alarm-setting tasks in a simple way. Instead of depending on a graphical interface, the user works with a text menu to add, view, remove, and switch alarms on or off.

The interesting part of the project is that the program does not stop at accepting an alarm time. A separate background thread keeps checking the clock while the main menu is still available. When an enabled alarm reaches its scheduled time, the program triggers the alarm and plays a sound.

2. Problem Statement

A basic alarm needs to do two things reliably: remember the time chosen by the user and notice when that time has arrived. For a beginner programmer, this also creates a useful practical problem involving user input, time calculations, program state, and background processing.

The aim of this project is to build a lightweight Python alarm clock that makes these tasks manageable through a simple command-line interface. The project also gives practical experience with modular programming and thread-based monitoring.

3. Functional Requirements

- The user can view the alarms that are currently configured.
- The user can add a new alarm by entering a time, with an optional label and daily repeat option.
- The user can remove an existing alarm.
- The user can turn an alarm on or off without deleting it.
- The user can check the next expected firing time.
- The program keeps watching the current time and triggers an enabled alarm when its time arrives.
- The program can be closed safely and save its configuration.

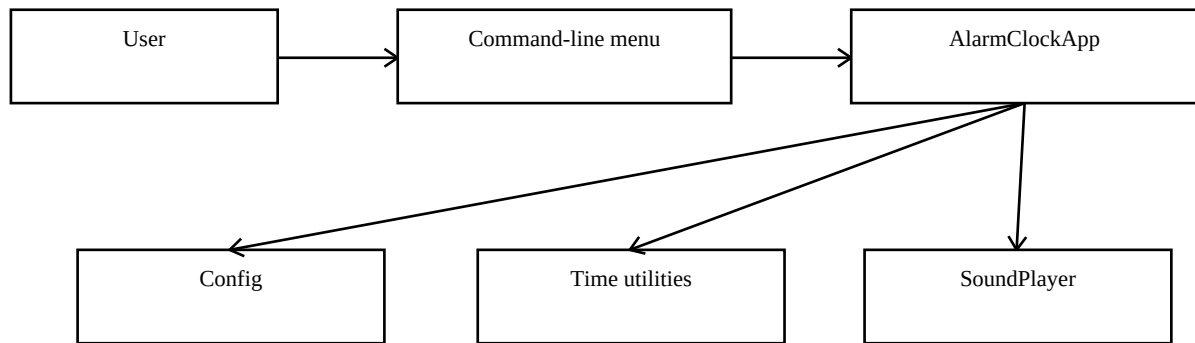
4. Non-functional Requirements

- Easy to use: the menu uses clear numbered choices instead of requiring complex commands.
- Responsive enough for a command-line application: the clock is checked periodically while the user can continue using the menu.
- Reliable: the program keeps track of alarms that have already fired for the day.
- Maintainable: related work such as configuration, sound playback, and time formatting/parsing is kept in separate components.
- Error-aware: invalid menu choices and interruptions are handled instead of being allowed to terminate the program unexpectedly.
- Lightweight: the monitoring loop waits between checks instead of continuously consuming CPU.

5. System Architecture

The project uses a simple layered/component approach. The user interacts with the command-line menu, while AlarmClockApp acts as the main controller. Supporting components handle configuration, sound,

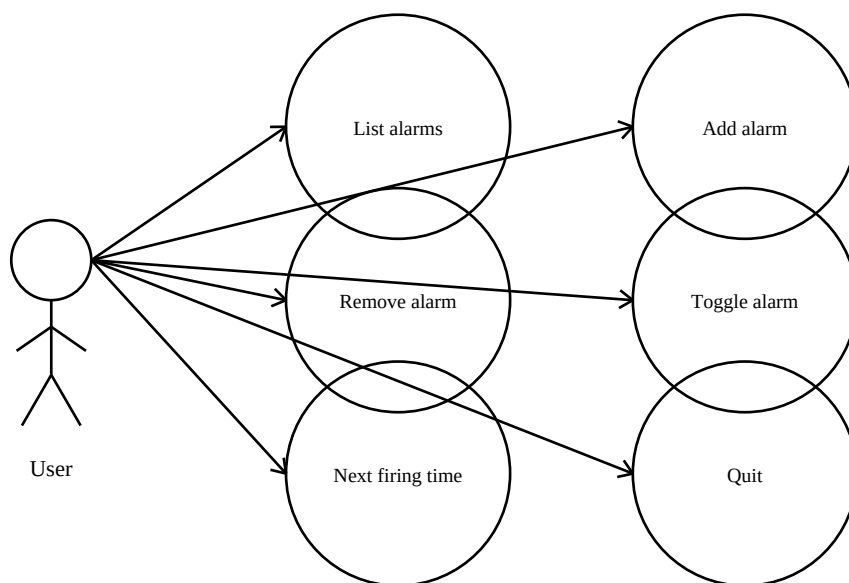
and time-related operations. At the same time, a background monitor checks whether an alarm needs to fire.



Background monitor checks the clock and triggers due alarms

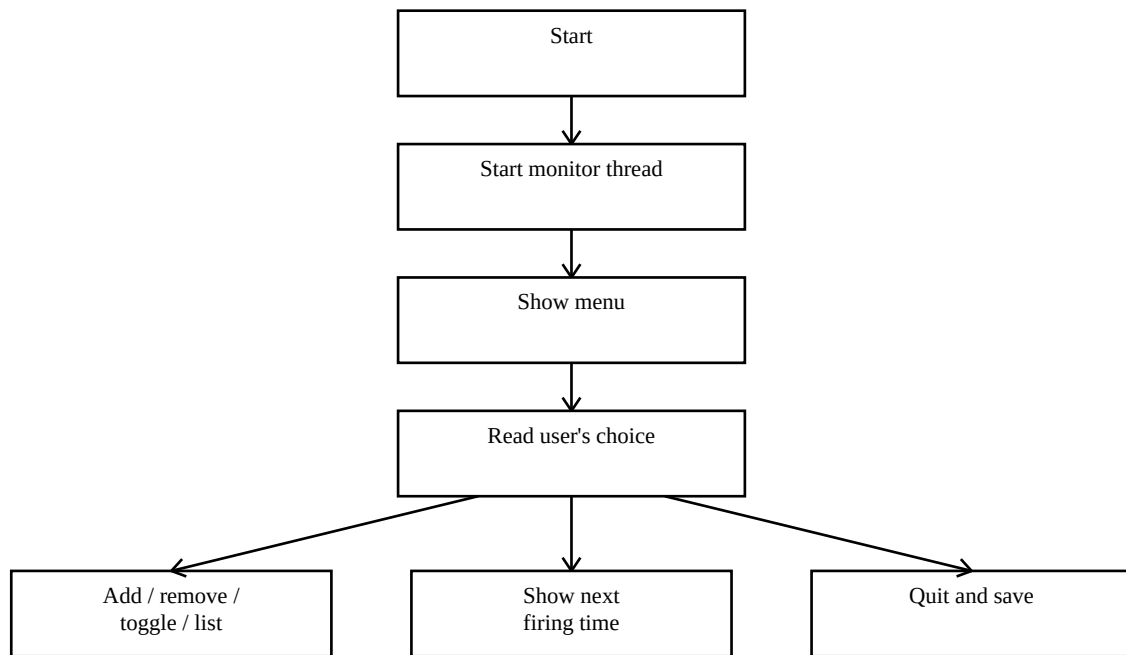
6. Design Diagrams

6.1 Use Case Diagram



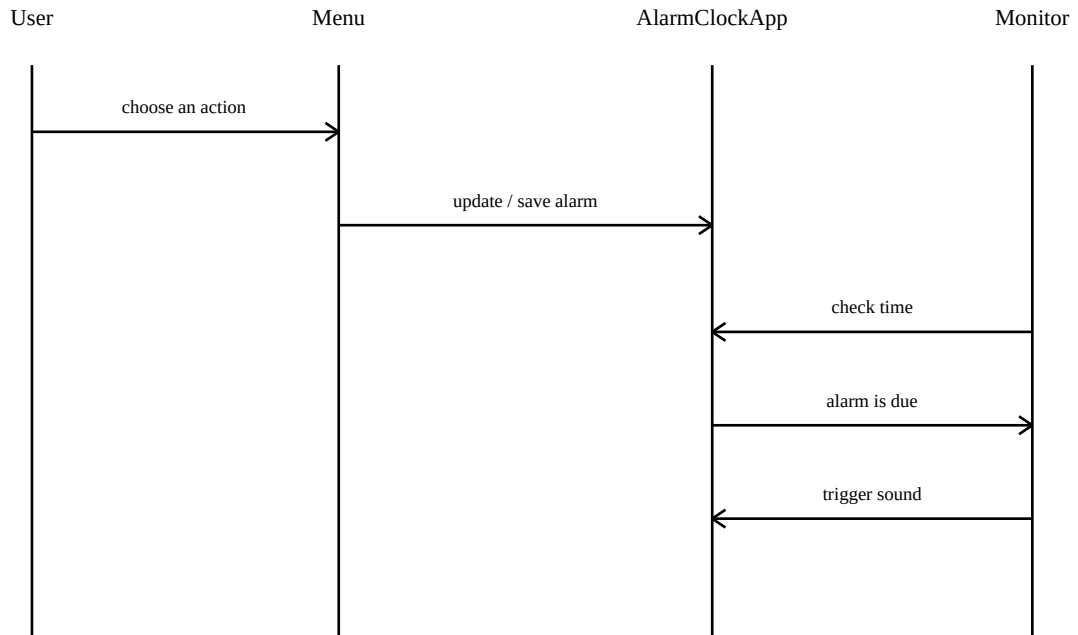
The user is the only main actor. The available actions are based directly on the application's menu: listing alarms, adding one, removing one, toggling it, checking the next firing time, and quitting.

6.2 Workflow Diagram



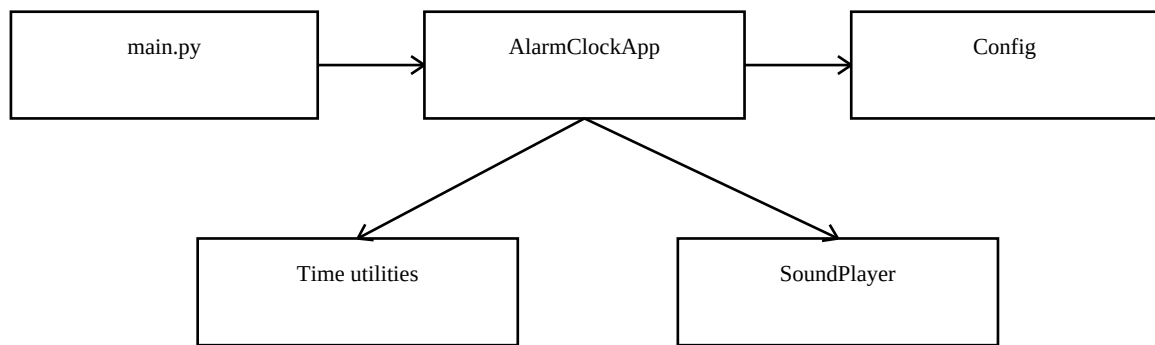
The program starts, launches the monitoring thread, displays the menu, carries out the selected operation, and keeps returning to the menu until the user quits.

6.3 Sequence Diagram



The sequence gives a simplified view of how an alarm can be added and how the background monitor later detects and triggers it.

6.4 Class / Component Diagram



The diagram shows the main entry point and the supporting components used by the application.

6.5 ER Diagram

An ER diagram is not included because this project does not use a relational database. Alarm information is handled through the application's configuration mechanism.

7. Design Decisions & Rationale

- A command-line interface was chosen because it keeps the project simple and lets the core alarm logic remain the focus.
- AlarmClockApp was used to keep the main application state and alarm operations together instead of putting everything into one long block of code.
- A background thread was used so the application can keep checking the time while the user is still interacting with the menu.
- Configuration, sound, and time utilities are separated from the main controller so each part has a clearer responsibility.
- The program records alarms that have already fired during the day, helping prevent the same scheduled occurrence from being triggered repeatedly.

8. Implementation Details

The main entry point imports Python's datetime and threading modules along with the project's configuration, sound, and time utility components. AlarmClockApp creates the required objects and starts a background monitoring thread.

The menu gives the user six choices: list alarms, add an alarm, remove an alarm, toggle an alarm, show the next firing time, or quit. Adding an alarm involves parsing the entered time, accepting an optional label, and setting whether it should repeat daily.

The monitoring part checks the current time at regular intervals. When it finds an enabled alarm whose scheduled time has arrived, it triggers the alarm and uses the sound component to play the ringtone. The program also handles interruption and operating-system errors during monitoring.

9. Screenshots / Results

This section should contain screenshots taken from the student's actual run of the program. The following screenshots are recommended:

- Main menu
- Adding an alarm
- List of configured alarms
- Toggling an alarm on or off
- Next firing time
- Alarm notification / sound result
- Final GitHub repository

Actual screenshots should be inserted before final submission. They are intentionally not fabricated in this report.

10. Testing Approach

Test	Expected result
Add a valid alarm	The alarm is added and can be seen in the list.
Enter an invalid time	The input is handled without crashing the application.
Remove an alarm	The selected alarm disappears from the configured list.
Toggle an alarm	The alarm changes between enabled and disabled.
Check next firing time	The program shows the next scheduled time when an alarm is available.
Wait for an alarm	The monitor detects the scheduled time and triggers the sound.
Quit the program	The application exits and saves its configuration.

These tests should be performed on the final version of the project, and the observed results can be mentioned alongside the screenshots.

11. Challenges Faced

- Making the program check the clock continuously without freezing the command-line menu.
- Working with time input and calculating when an alarm should fire next.
- Making sure one alarm occurrence does not trigger over and over during repeated checks.
- Keeping the main application readable while moving configuration, sound, and time-related work into separate components.
- Handling keyboard interruption and other runtime errors while the monitoring thread is active.

12. Conclusion

This project started with a simple idea—make an alarm clock in Python—but it brings together several useful programming concepts. The application accepts user input, manages alarm state, works with time, saves configuration, runs a background monitoring task, and produces a sound notification.

Overall, the project is a practical example of how a small Python program can be broken into separate responsibilities instead of keeping all the logic in one place. It also provides a good base for future

improvements such as a graphical interface, more flexible repeat schedules, multiple sound choices, or additional notification methods.